

Terraform Local values

What are Local Values in Terraform?

Local values in Terraform are like temporary variables.

You use them to store a value (like a string, number, or list) so that you can reuse it many times in your code instead of writing it again and again.

Why Use Local Values?

- To avoid repeating the same value or expression multiple times.
- To make your code **cleaner** and **easier to understand**.
- To give a name to a calculated value.

Simple Example:

Without locals:

```
resource "aws_instance" "web1" {  
  
    instance_type = "t2.micro"  
  
    tags = {  
  
        Name = "web-server"  
  
    }  
}  
  
resource "aws_instance" "web2" {  
  
    instance_type = "t2.micro"  
  
    tags = {  
  
        Name = "web-server"  
  
    }  
}
```

You're repeating the same values ("t2.micro", "web-server") in both places.

With **locals**:

```
locals {  
  
    my_instance_type = "t2.micro"  
  
    my_tag_name      = "web-server"  
  
}  
  
resource "aws_instance" "web1" {  
  
    instance_type = local.my_instance_type  
  
    tags = {  
  
        Name = local.my_tag_name  
  
    }  
  
}  
  
resource "aws_instance" "web2" {  
  
    instance_type = local.my_instance_type  
  
    tags = {  
  
        Name = local.my_tag_name  
  
    }  
  
}
```

Now, you're using the local values:

- local.my_instance_type
- local.my_tag_name

If you want to change the instance type or tag, you change it **only in one place** (inside **locals**), and it updates everywhere.



How to Declare Local Values?

```
locals {  
  
    region_name = "us-east-1"  
  
    project_name = "myproject"  
  
    full_name = "${local.project_name}-${var.environment}"  
  
}
```

Then you can use it like:

```
provider "aws" {  
  
    region = local.region_name  
  
}
```



Use Cases

Situation	Local value helps you by...
Repeated value	Write it once, use many times
Complex calculation	Do the calculation once, reuse it
Dynamic expressions	Store expressions for use in many places



Final Summary

- **locals** is a block used to define reusable values.
- Use **local.<name>** to access the local value.
- Makes your Terraform code DRY (Don't Repeat Yourself) and easy to maintain.

VARIABLE vs LOCAL in Terraform

Feature	variable	local
 Used For	Getting input from the user or external files	Storing intermediate or reusable values (like a temp var)
 Defined In	variable "name" {}	locals { name = value }
 Value Set By	User (in terraform.tfvars, CLI, or env vars)	Defined directly in the code
 Reusability	Use across modules as external input	Use internally in one file or module
 Can Change at Runtime?	Yes – based on input	No – always fixed once declared
 Typical Use Case	User inputs like region, environment, instance type	Calculated names, common tags, derived values

Example:

variables.tf:

```
variable "environment" {  
  
  type = string  
  
}
```

terraform.tfvars:

```
environment = "dev"
```

main.tf:

```
locals {  
  
  project_name = "myapp"  
  
  full_name = "${local.project_name}-${var.environment}"  
  
}
```

```
output "app_name" {  
    value = local.full_name  
}
```

 **In this example:**

Expression	Meaning
var.environment	User provides this (e.g., "dev")
local.project_name	Hardcoded internal value ("myapp")
local.full_name	Combines both (myapp-dev)

Simple Analogy:

- **variable** = User input field ("Please tell me your name")
- **local** = Scratchpad calculation ("Let me combine your name with today's date")

When to Use What?

- Use ****variable**** when:
 - You want users to provide a value (e.g., region, environment).
 - The value can change between deployments.
- Use ****local**** when:
 - You want to reuse a value.
 - You want to calculate something from variables (e.g., full name, tags).
 - The value should not be changed externally.

Full working example with **locals**, **variables**, and a resource block together

✓ **variables**

✓ **locals**

✓ a resource block (**aws_instance**)

✓ and using **local** values inside the resource

✓ **Step-by-Step Terraform Example**

Folder Structure

```
terraform-project/  
├── main.tf  
├── variables.tf  
└── terraform.tfvars
```

variables.tf – Define input variables

```
variable "environment" {  
  description = "The environment name (e.g. dev, prod)"  
  type        = string  
}
```

```
variable "instance_type" {  
  description = "EC2 instance type"  
  type        = string  
}
```

terraform.tfvars – Provide values for variables

```
environment = "dev"  
instance_type = "t3.micro"
```

main.tf – Use locals and create resource

```
provider "aws" {  
  region = "us-east-1"  
}
```

Define reusable values

```
locals {  
  project_name = "myapp"  
  full_name    = "${local.project_name}-${var.environment}"  
  tags = {  
    Project    = local.project_name  
    Environment = var.environment  
    Owner      = "TeamA"  
  }  
}
```

```
resource "aws_instance" "web_server" {  
  ami          = "ami-0c55b159cbfafa1f0" # Amazon Linux 2 AMI (replace with latest)  
  instance_type = var.instance_type
```

```
  tags = local.tags
```

Optional name tag using locals

```
tags_all = {  
  Name = local.full_name  
}  
}
```

What is Happening?

Component	Purpose
<code>`variables.tf`</code>	Declares input variables
<code>`terraform.tfvars`</code>	Supplies values for the input variables
<code>`locals` block</code>	Combines and simplifies values like <code>`project_name`</code> , <code>`tags`</code>
<code>`aws_instance`</code>	Creates an EC2 instance using both <code>`var`</code> and <code>`local`</code> values

✓ Output Example:

The EC2 instance will be:

- Type: **t3.micro**
- Name Tag: **myapp-dev**
- Other Tags: **Project=myapp, Environment=dev, Owner=TeamA**